



# CS 5/7320 Artificial Intelligence

## Intelligent Agents AIMA Chapter 2

---

Slides by Michael Hahsler  
based on slides by Svetlana Lazepnik  
with figures from the AIMA textbook.



This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/).

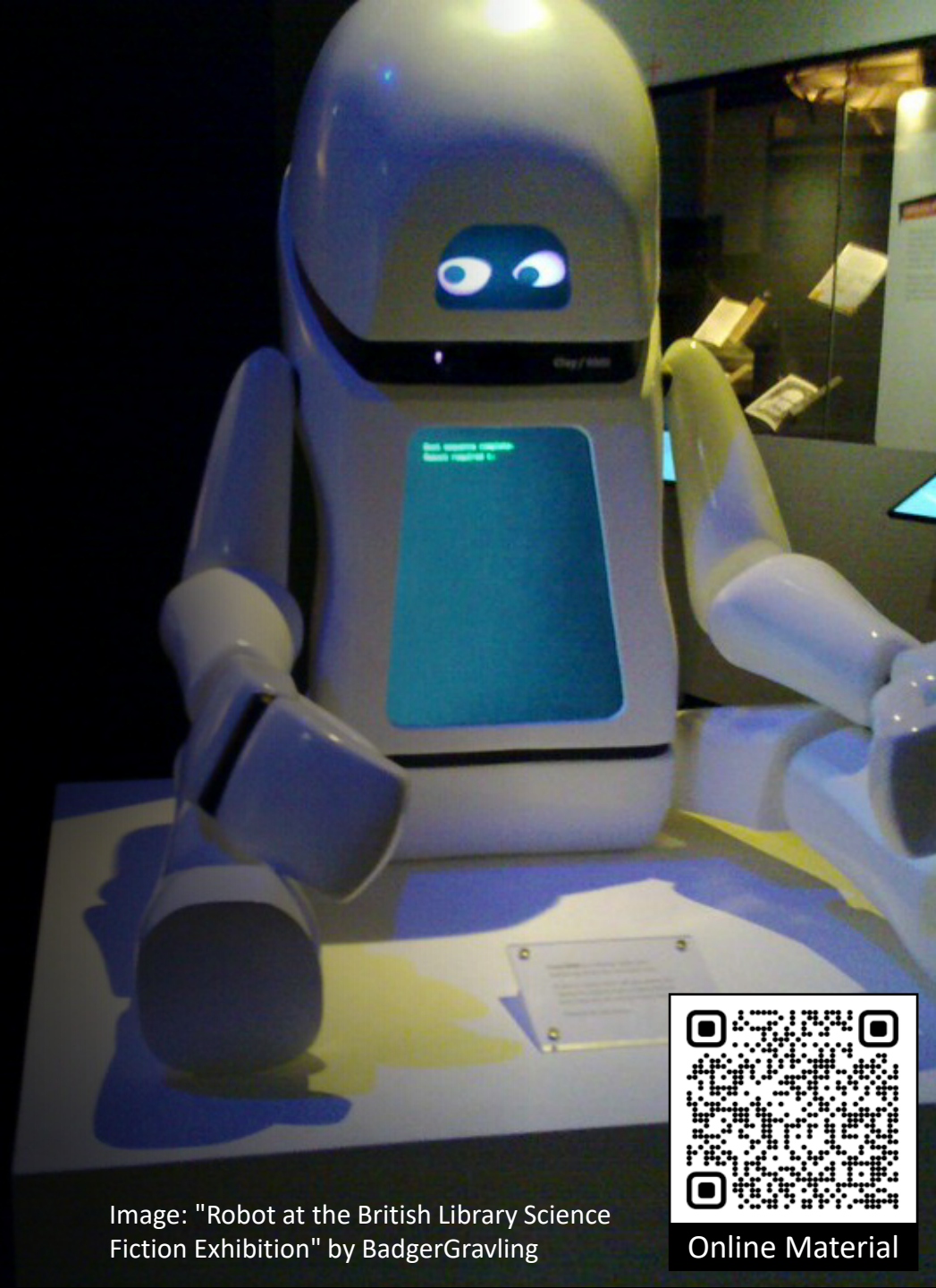
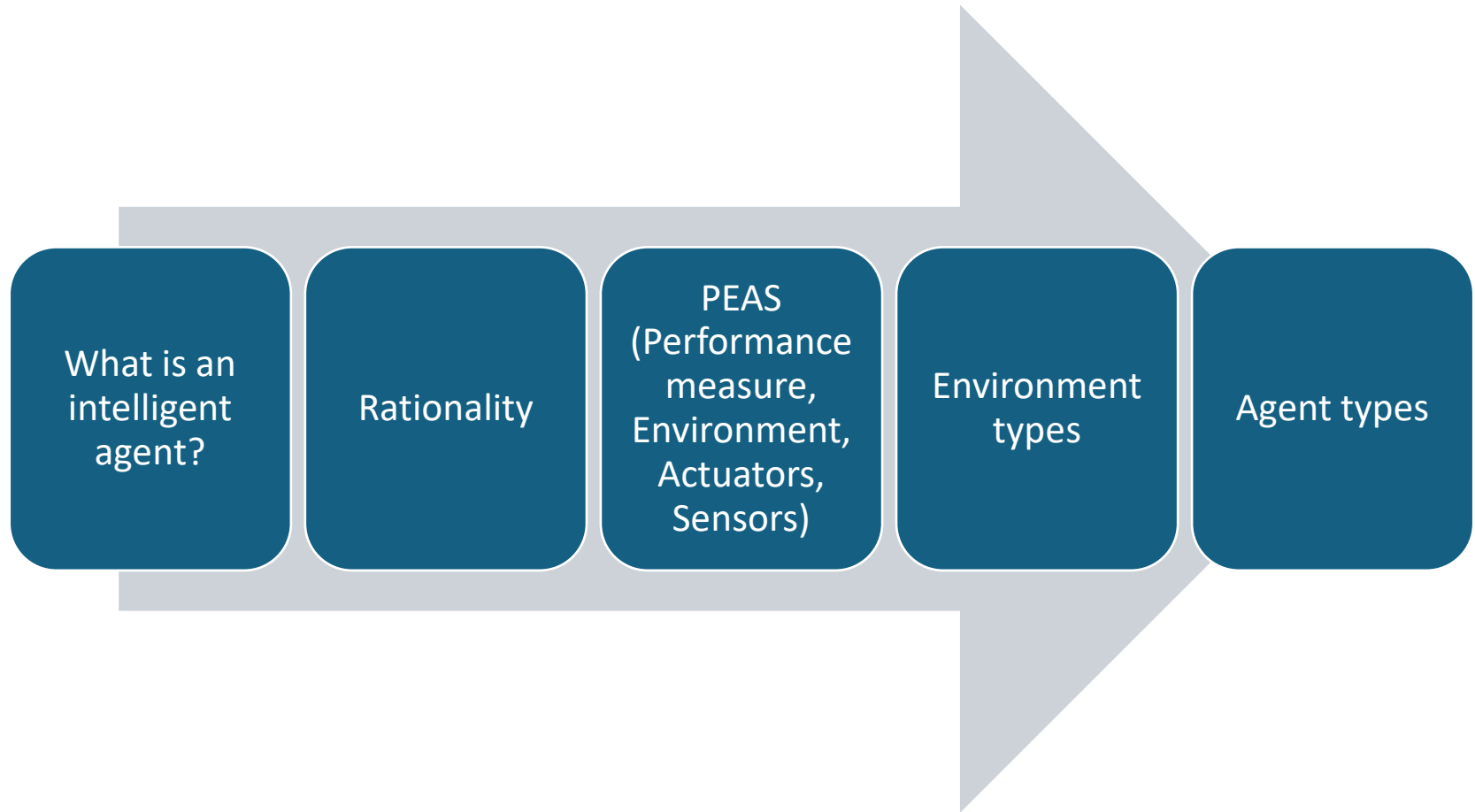


Image: "Robot at the British Library Science Fiction Exhibition" by BadgerGravling

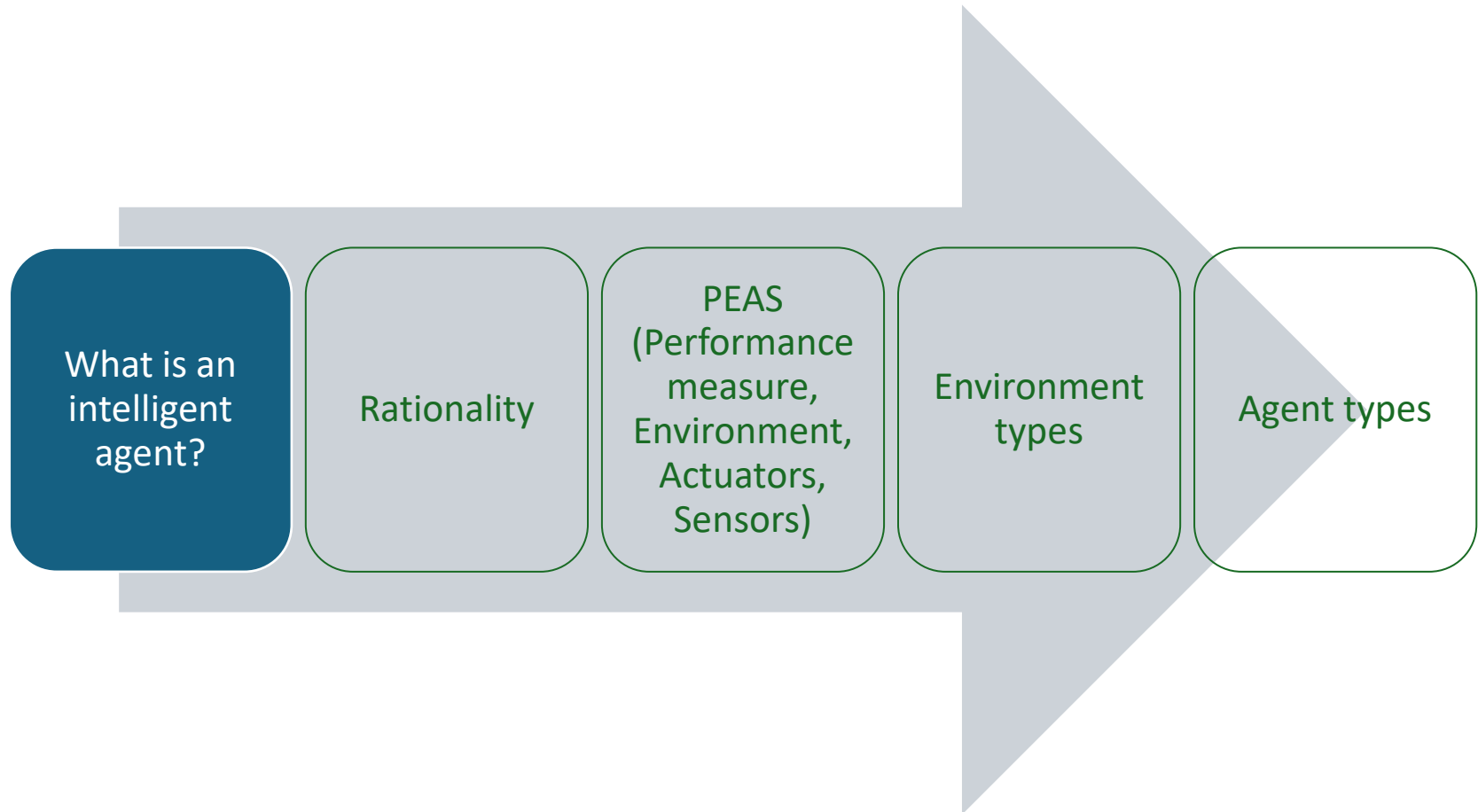


Online Material

# Outline

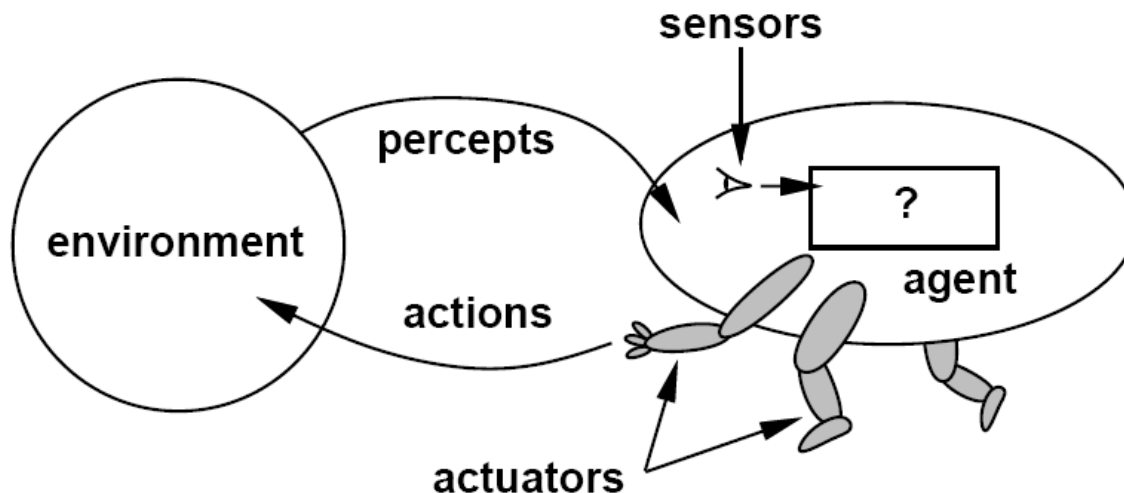


# Outline: What is an Intelligent Agent



# What is an Agents?

- An **agent** is anything that can be viewed as **perceiving** its **environment** through **sensors** and **acting** upon that environment through **actuators**.

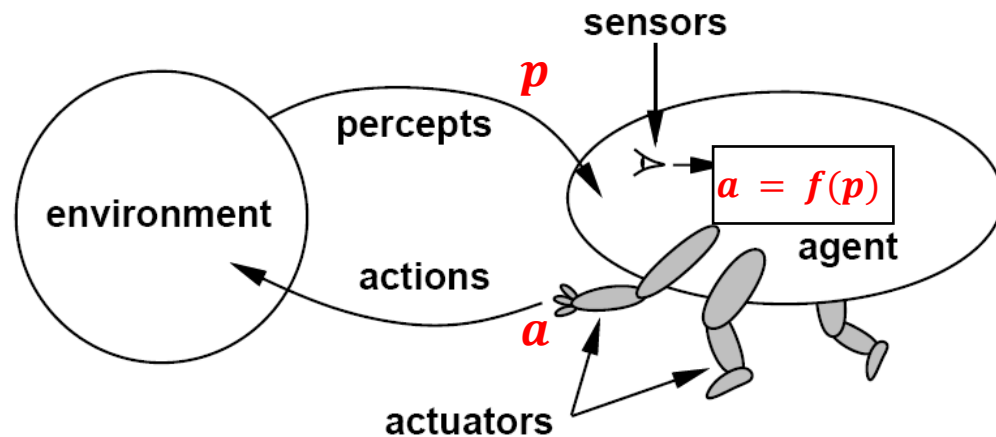


- **Control theory:** A **closed-loop control system** (= feedback control system) is a set of mechanical or electronic devices that automatically regulate a process variable to a desired state or set point without human interaction. The agent is called a controller.
- **Softbot:** Agent is a software program that runs on a host device.

# Agent Function and Agent Program

The **agent function** maps from the set of all possible *percept sequences*  $P^*$  to the *set of actions*  $A$  formulated as an abstract mathematical function.

$$f : P^* \rightarrow A$$



The **agent program** is a concrete implementation of this function for a given physical system.

Agent = architecture (hardware) + agent program (implementation of  $f$ )



- Sensors
- Memory
- Computational power

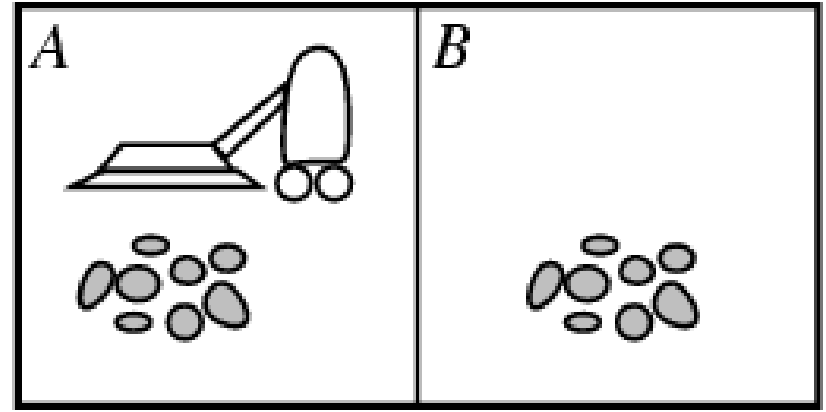
# Example: Vacuum-cleaner World

- **Percepts:**

Location and status,  
e.g., [A, Dirty]

- **Actions:**

Left, Right, Suck, NoOp



Most recent  
Percept  $p$

Agent function:  $f : P^* \rightarrow A$

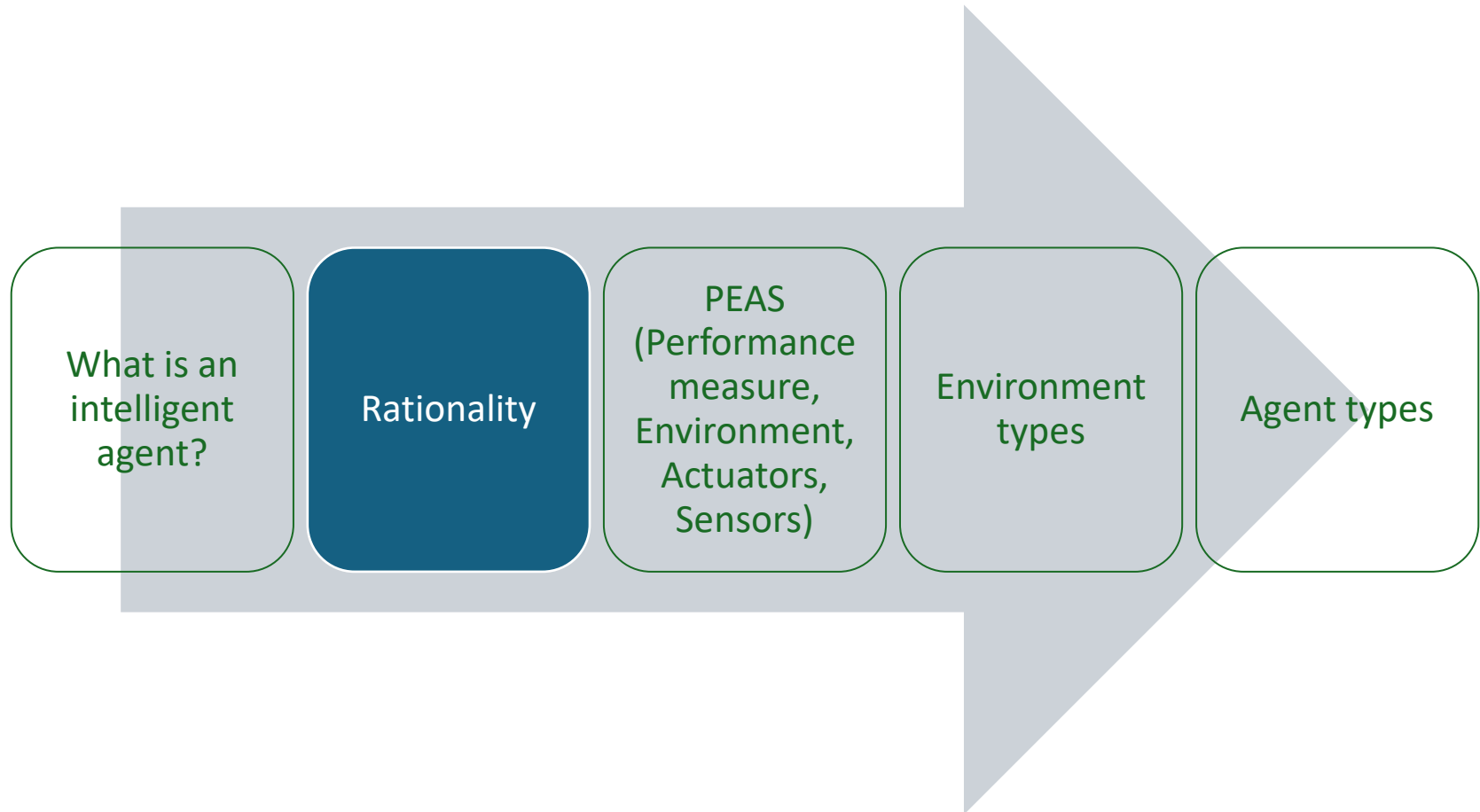
| Percept Sequence                   | Action |
|------------------------------------|--------|
| [A, Clean]                         | Right  |
| [A, Dirty]                         | Suck   |
| ...                                |        |
| [A, Clean], [B, Clean]             | Left   |
| ...                                |        |
| [A, Clean], [B, Clean], [A, Dirty] | Suck   |
| ...                                |        |

Implemented agent program:

```
function Vacuum-Agent( [location, status] )  
    returns an action  $a$   
  
    if status = Dirty then return Suck  
    else if location = A then  
        return Right  
    else if location = B then  
        return Left
```

**Problem:** This table can become infinitively large!

# Outline: Rationality



# Rational Agents: What is Good Behavior?

Foundation from normative moral theory and economics:

- **Consequentialism:** Evaluate actions by their consequences.
- **Utilitarianism:** Maximize happiness and well-being.

Definition of a rational agent:

*“For each possible percept sequence, a rational agent should select an **action** that **maximizes its expected performance measure**, given the evidence provided by the **percept sequence** and the **agent’s built-in knowledge**.”*

- **Performance measure:** An *objective* criterion for success of an agent's behavior (often called utility function or reward function).
- **Expectation:** Outcome averaged over all possible situations that may arise.

**Rule:** Pick the action that maximizes the expected utility

$$a = \operatorname{argmax}_{a \in A} \mathbb{E}(U \mid a)$$



# Rational Agents

**Rule:** Pick the action that maximizes the expected utility

$$a = \operatorname{argmax}_{a \in A} \mathbb{E}(U \mid a)$$

This means:

- **Rationality is an ideal** – it implies that no one can build a better agent
- **Rationality  $\neq$  Omniscience** – rational agents can make mistakes if percepts and knowledge do not suffice to make a good decision
- **Rationality  $\neq$  Perfection** – rational agents maximize **expected** outcomes not actual outcomes
- **It is rational to explore and learn** – i.e., use **percepts** to supplement prior knowledge and become autonomous
- **Rationality is often bounded** by available memory, computational power, available sensors, etc.

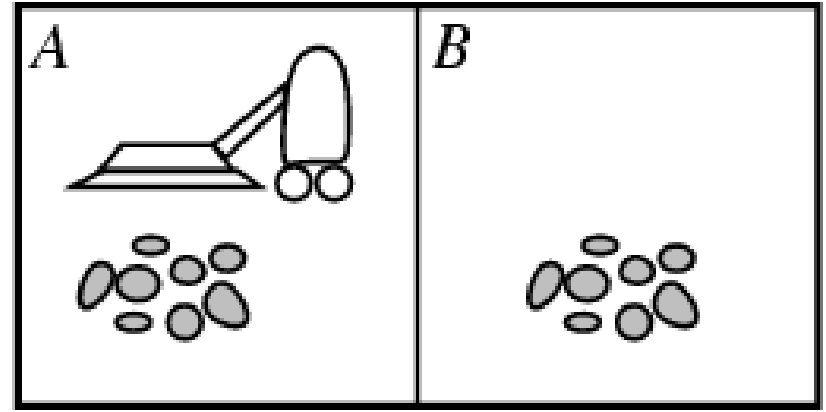
## Example: Performance Measure for the Vacuum-cleaner World

- **Percepts:**

Location and status,  
e.g., [A, Dirty]

- **Actions:**

Left, Right, Suck, NoOp



Agent function:

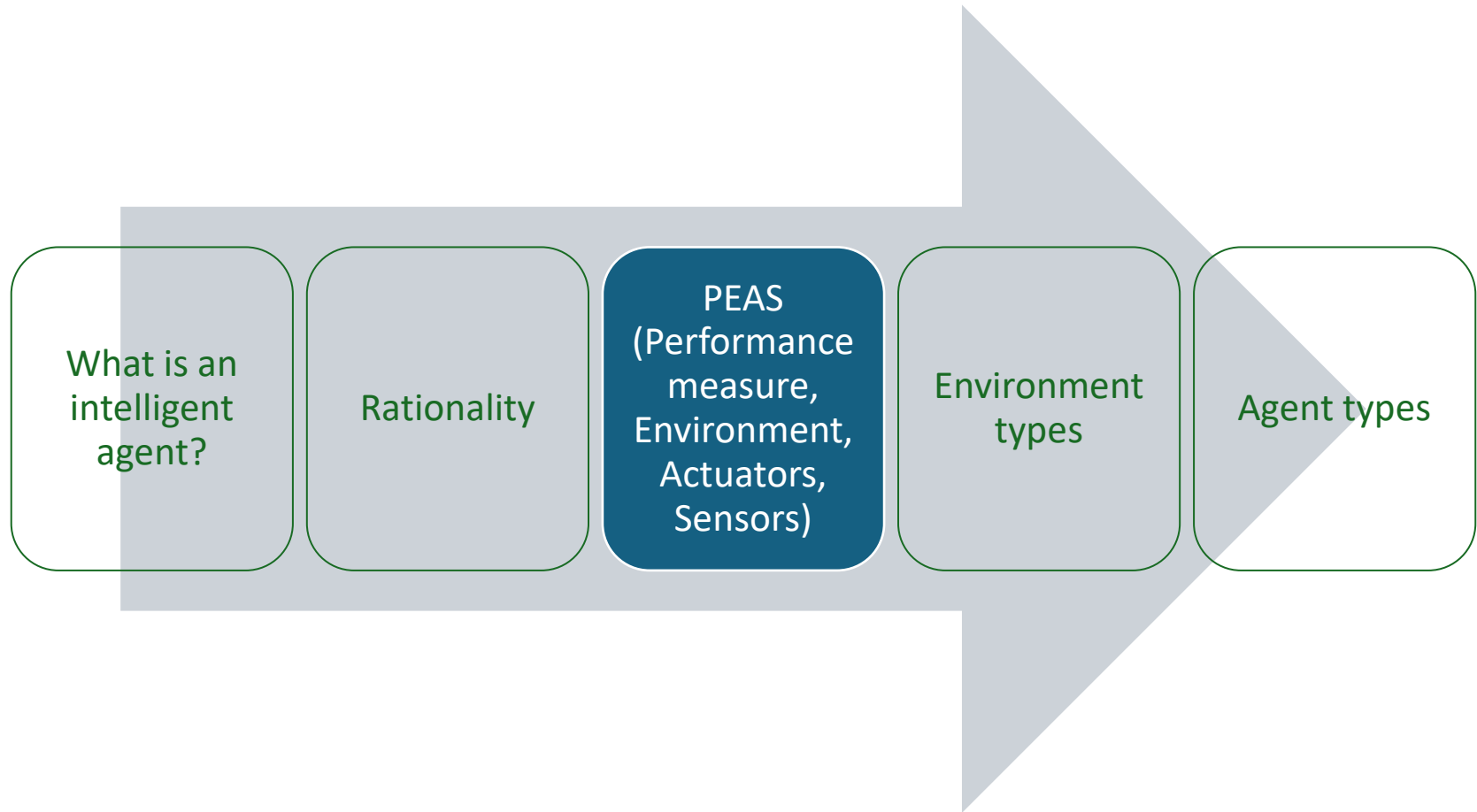
| <u>Percept Sequence</u> | <u>Action</u> |
|-------------------------|---------------|
| [A, Clean]              | Right         |
| [A, Dirty]              | Suck          |
| ...                     |               |
| [A, Clean], [B, Clean]  | Left          |
| ...                     |               |

Implemented agent program:

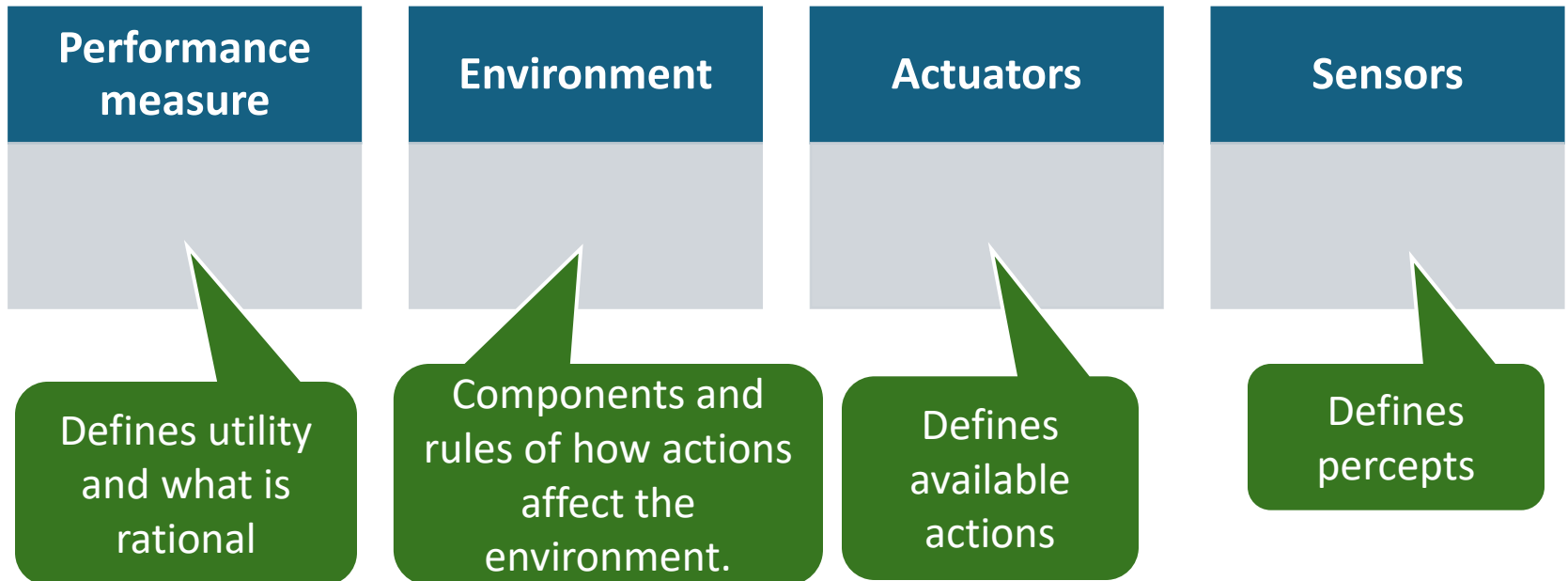
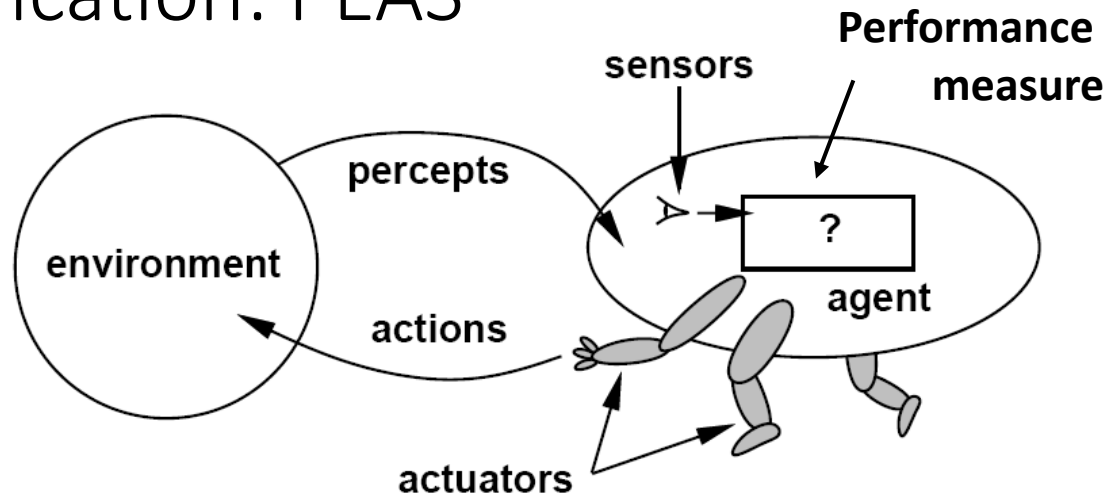
```
function Vacuum-Agent( [location, status] )  
    returns an action  
  
    if status = Dirty then return Suck  
    else if location = A then return Right  
    else if location = B then return Left
```

**What could be a performance measure?**  
**Is this agent program rational?**

# Outline: PEAS



# Problem Specification: PEAS



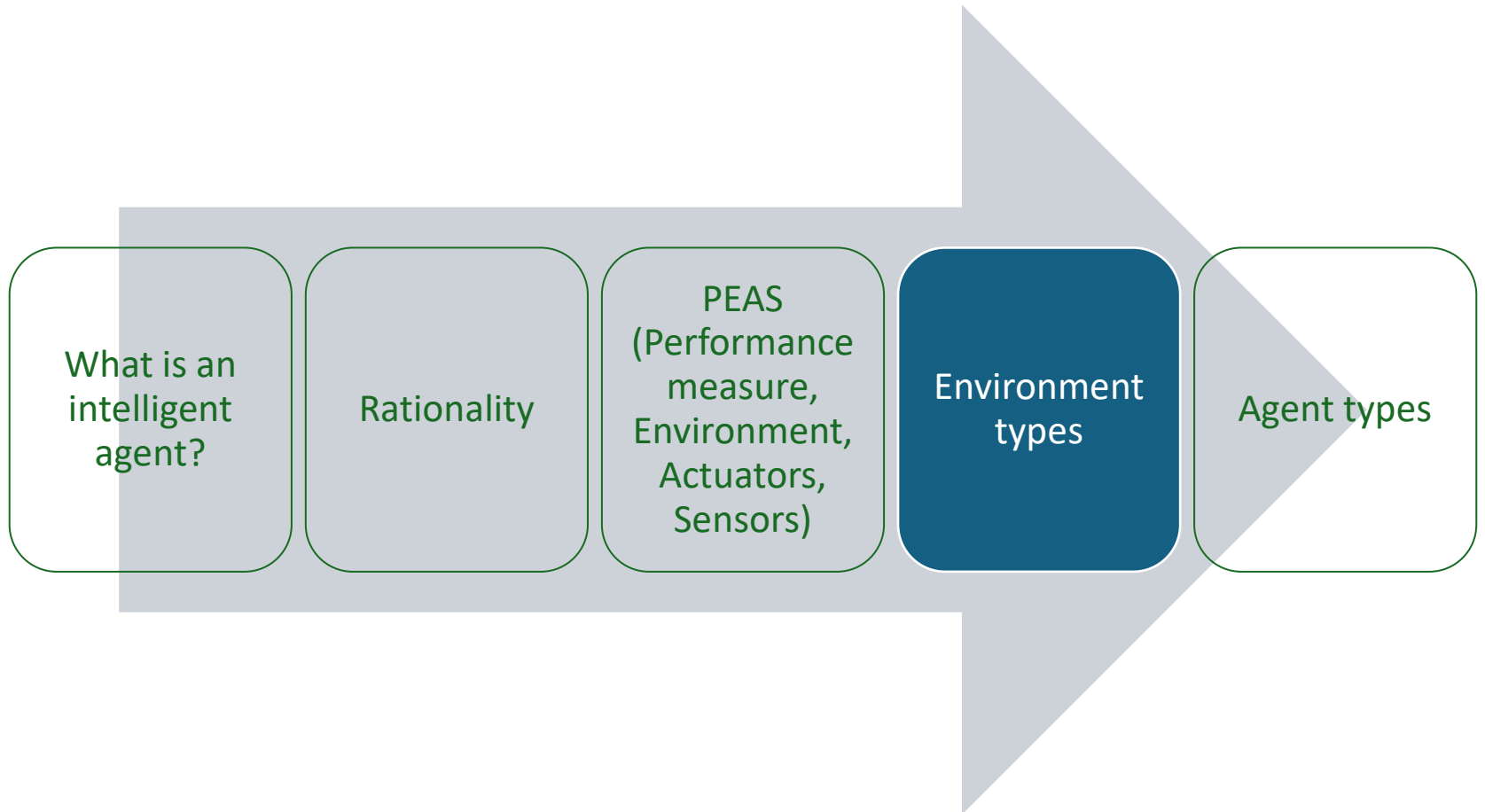
# Example: Automated Taxi Driver

| Performance measure                                                                                                                         | Environment                                                                                                               | Actuators                                                                                                                              | Sensors                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Safe</li><li>• fast</li><li>• legal</li><li>• comfortable trip</li><li>• maximize profits</li></ul> | <ul style="list-style-type: none"><li>• Roads</li><li>• other traffic</li><li>• pedestrians</li><li>• customers</li></ul> | <ul style="list-style-type: none"><li>• Steering wheel</li><li>• accelerator</li><li>• brake</li><li>• signal</li><li>• horn</li></ul> | <ul style="list-style-type: none"><li>• Cameras</li><li>• sonar</li><li>• speedometer</li><li>• GPS</li><li>• Odometer</li><li>• engine sensors</li><li>• keyboard</li></ul> |

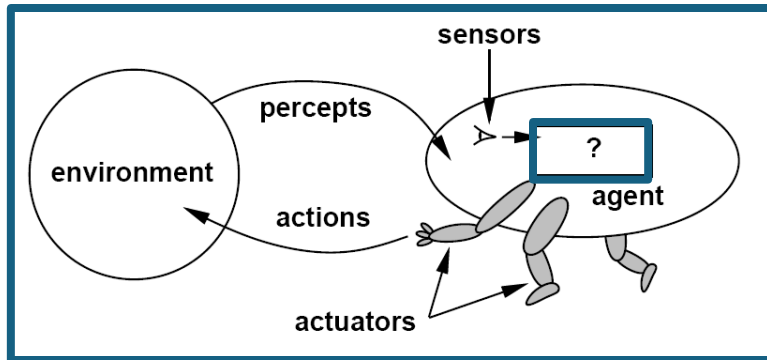
# Example: Spam Filter

| Performance measure                                                                                     | Environment                                                                                     | Actuators                                                                                      | Sensors                                                                                                              |
|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Accuracy: Minimizing false positives, false negatives</li></ul> | <ul style="list-style-type: none"><li>• A user's email account</li><li>• email server</li></ul> | <ul style="list-style-type: none"><li>• Mark as spam</li><li>• delete</li><li>• etc.</li></ul> | <ul style="list-style-type: none"><li>• Incoming messages</li><li>• other information about user's account</li></ul> |

# Outline: Environment Types

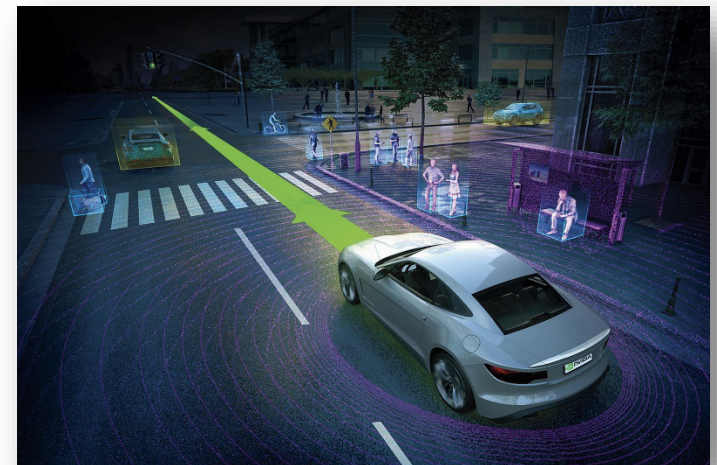


# The Environment



- We typically consider everything outside the agent function (the agent's brain) as the agent's environment.
- This means that the sensors and actuators are part of the environment.
- The agent function receives already preprocessed percepts and acts by issuing high-level instructions to the actuators.

Examples:





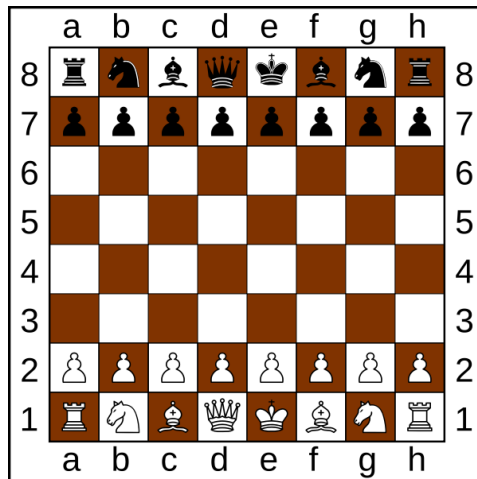
# Environment Types: Observability

**Fully observable:** The agent's sensors give it access to the complete state of the environment. The agent can “see” the whole environment.

**VS.**

**Partially observable:** The agent cannot see all aspects of the environment. E.g., it can't see through walls

Examples:



Chess

- All Pieces visible
- Full information



Poker

- Some cards are not known (e.g., other player's cards)

# Environment Types: Stochasticity

**Deterministic:** The environment follows fixed rules. Actions have always the same results. **vs.**

**Stochastic:** The environments reacts to actions with some randomness. The same action may have different results when tried multiple times.

Examples:



Tic-tac-toe

- The game follows fixed rules without uncertainty

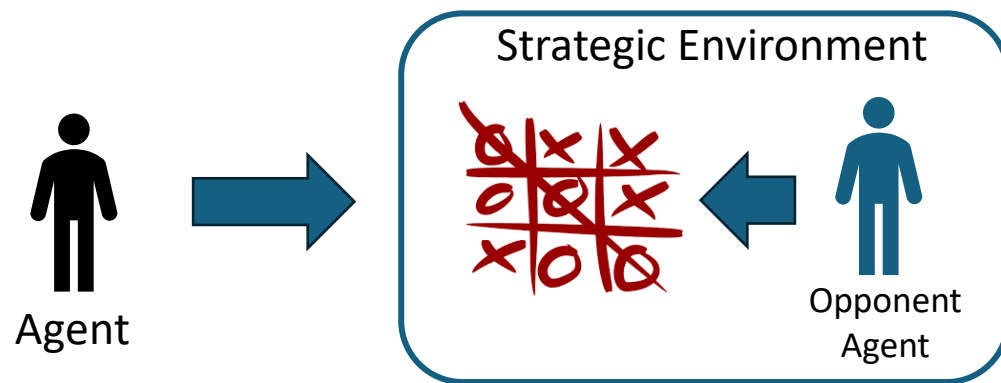


Games with uncertainty:

- Roll dice to move
- Pick a card from a stack

# Special Case of Stochastic Environments: Strategic Environment

- Standard stochastic environments include pure randomness.
- Strategic environments pursue their own objective. This typically means that the environment contains a second agent making decisions.
- Uncertainty comes from the unpredictable, interactive, and often adversarial choices of the other intelligent agents.



**Note:** While the game mechanics are deterministic, considering the opponent as part of the environment makes the environment strategic (stochastic)!

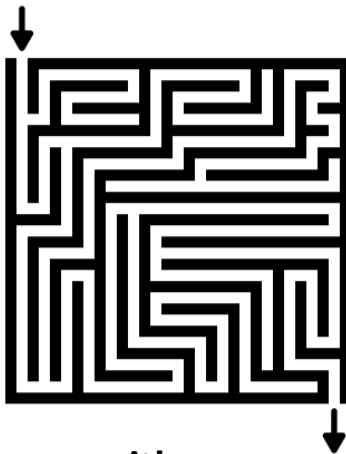
# Environment Types: Knowledge

**Known:** The agent knows the rules of the environment and can predict the (chance of the) outcome of actions.

**vs.**

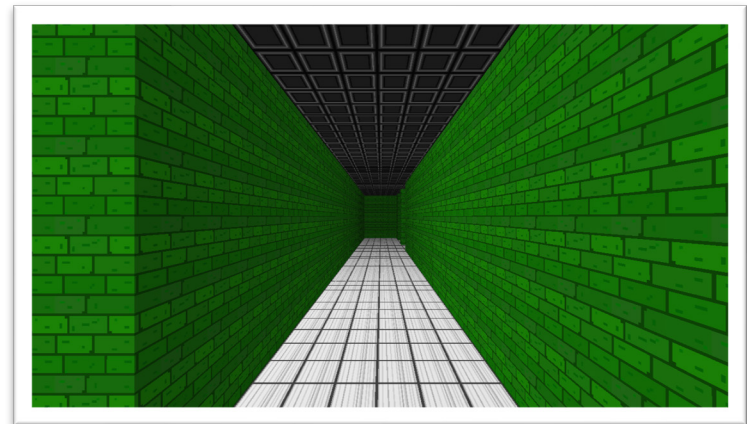
**Unknown:** The agent does not know the rules of the environment and cannot predict the outcome of actions.

Examples:



Solving a maze **with** a map

- The agent knows in every position where actions (North, East, etc) lead.
- The map can be used for planning.



Solving a maze **without** a map

- The agent does not know where all the corridors lead.
- The agent needs to explore.

# More Environment Types

**Static:** The environment is **not** changing while agent is deliberating.

**Semidynamic:** the environment is static, but the agent's performance score depends on how fast it acts.

**Discrete:** The environment provides a fixed number of distinct percepts, actions, and environment states. Time can also evolve in a discrete or continuous fashion.

**Episodic:** Episode = a self-contained sequence of actions. **Short episodes** for a task that the agent performs repeatedly. What the agent does in one episode does not affect future episodes.

**Single agent:** A single agent operating in an environment.

**vs.**

**Dynamic:** The environment is changing while the agent is deliberating.

**vs.**

**Continuous:** Percepts, actions, state variables or time are continuous leading to an infinite state, percept or action space.

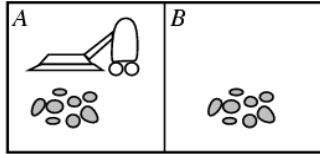
**vs.**

**Sequential:** Tasks are long, and actions taken now affect the outcomes later. The agent must consider the **long-term consequences** of its actions.

**vs.**

**Multi-agent:** Agent cooperate or compete in the same environment.

# Examples of Different Environments



Vacuum cleaner world



Scrabble



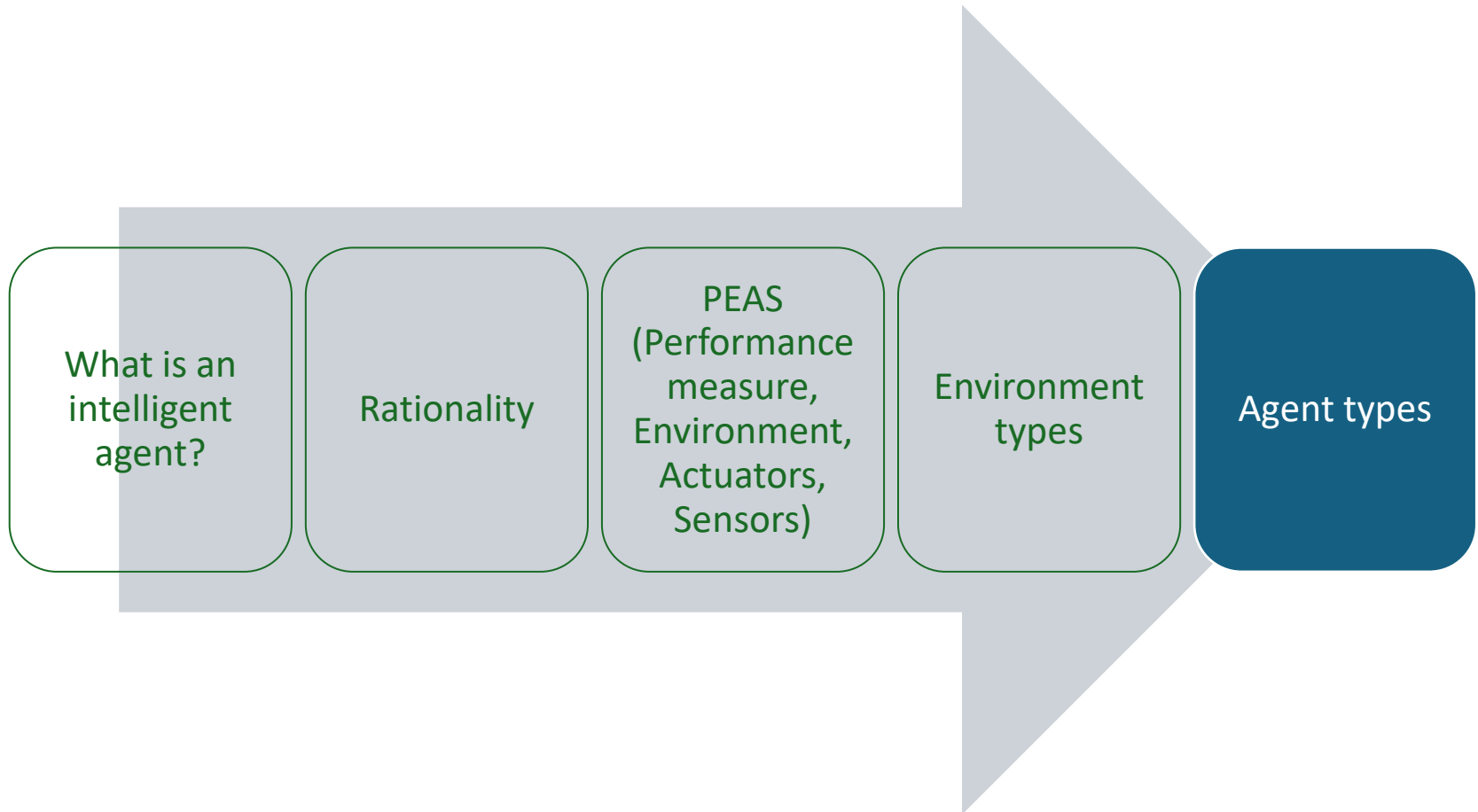
Driving a taxi

|               |               |              |            |
|---------------|---------------|--------------|------------|
| Observable    | Partially     | Partially    | Partially  |
| Deterministic | Deterministic | Stochastic   | Stochastic |
| Episodic?     | Episodic      | Sequential** | Sequential |
| Static        | Static        | Static       | Dynamic    |
| Discrete      | Discrete      | Discrete     | Continuous |
| Single agent  | Single        | Multi*       | Multi*     |

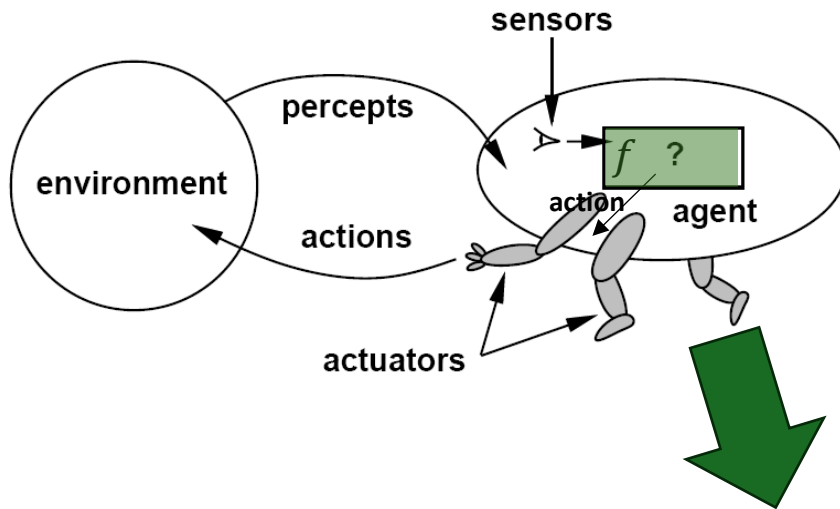
\* Can be models as a single agent problem with the other agent(s) in the environment.

\*\* A single game would be sequential environment. Multiple games could be also modeled as an episodic sequence of independent games.

# Outline: Agent Types



# Designing a Rational Agent

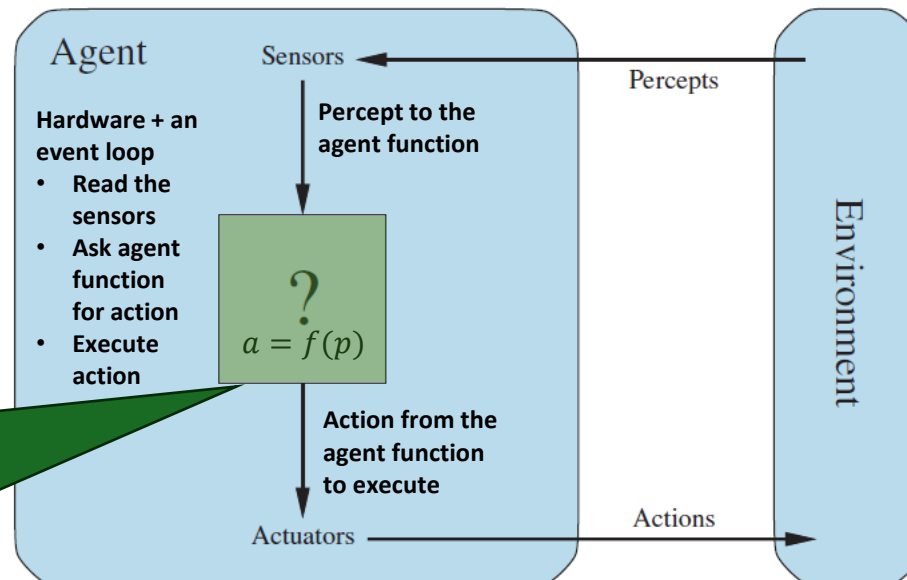


Remember the definition of a rational agent:

*“For each possible percept sequence, a rational agent should select an **action** that **maximizes its expected performance measure**, given the evidence provided by the **percept sequence** and the **agent’s built-in knowledge**.”*

**Agent Function**

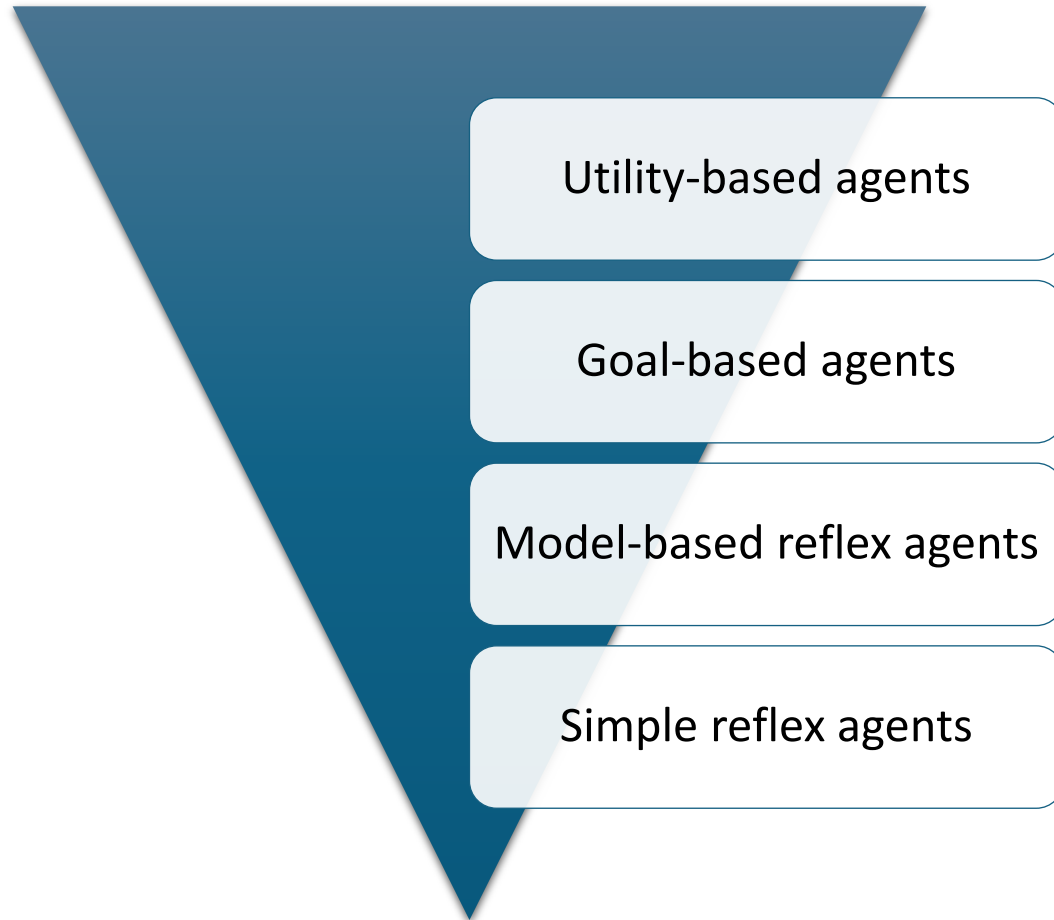
- Represents the “brain”
- Assess performance measure
- Remember percept sequence
- Built-in knowledge



**Important:** Everything outside the agent function represents the environment. This includes the physical robot, its sensors and its actuators, and event loop!

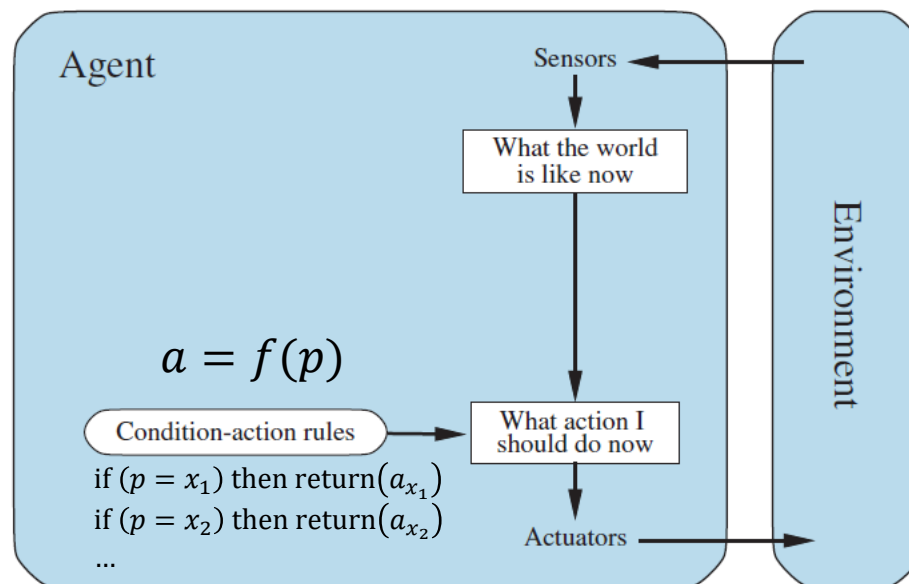


# Hierarchy of Agent Types



# Simple Reflex Agent

- Uses only built-in knowledge in the form of **rules** that select action only **based on the current percept**. This is typically very fast!
- The **agent does not know about the performance measure**! But well-designed rules can lead to good performance.
- The agent needs no memory and ignores all past percepts.

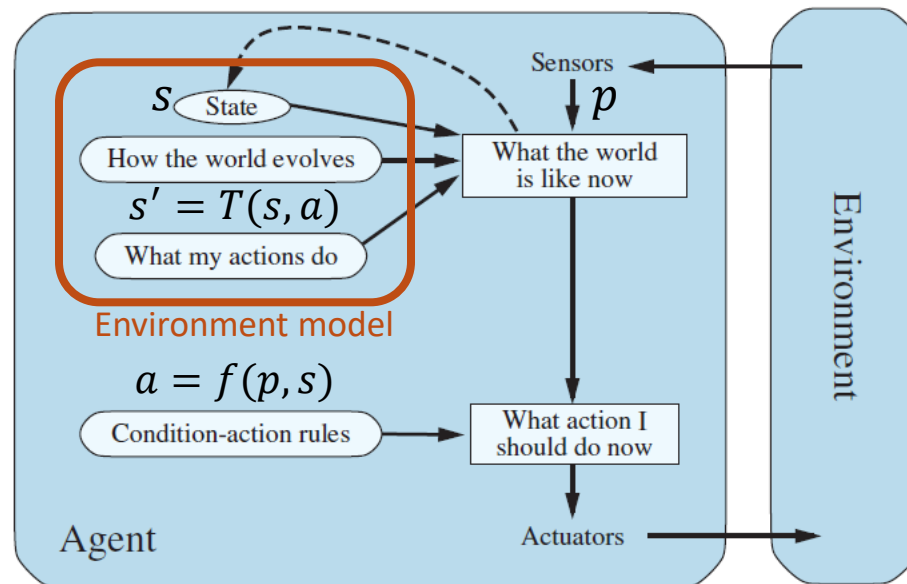


The interaction is a sequence:  $p_0, a_0, p_1, a_1, p_2, a_2, \dots, p_t, a_t, \dots$

**Example:** A simple vacuum cleaner that uses rules based on its current sensor input.

# Model-based Reflex Agent

- Maintains a **state variable** to keep track of aspects of the environment that cannot be currently observed. I.e., it has memory.
- It knows how the environment evolves over time and what its actions do (implemented as the **transition function**) to keep its state up-to-date.
- There is now **more information for the rules** to make better decisions.



**Note:**  $T$  is not directly used in the rules, but may affect rule design and the state update.

The interaction is a sequence:  $p_0, s_0, a_0, p_1, s_1, a_1, p_2, s_2, a_2, p_3, \dots, p_t, s_t, a_t, \dots$

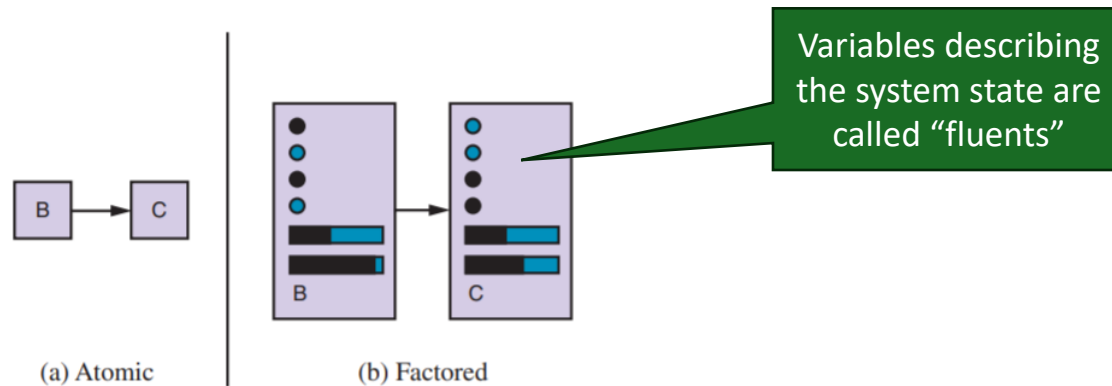
**Example:** A vacuum cleaner that remembers where it has already cleaned.

# State Representation

**Definition:** The state represents all that the agent knows about the environment and can use for choosing an action.

The representation can be:

- **Atomic:** Just a label for a black box. E.g., A, B
- **Factored:** A set of attribute values called fluents (because they model what can change). E.g., [location = left, status = clean, temperature = 75 deg. F]



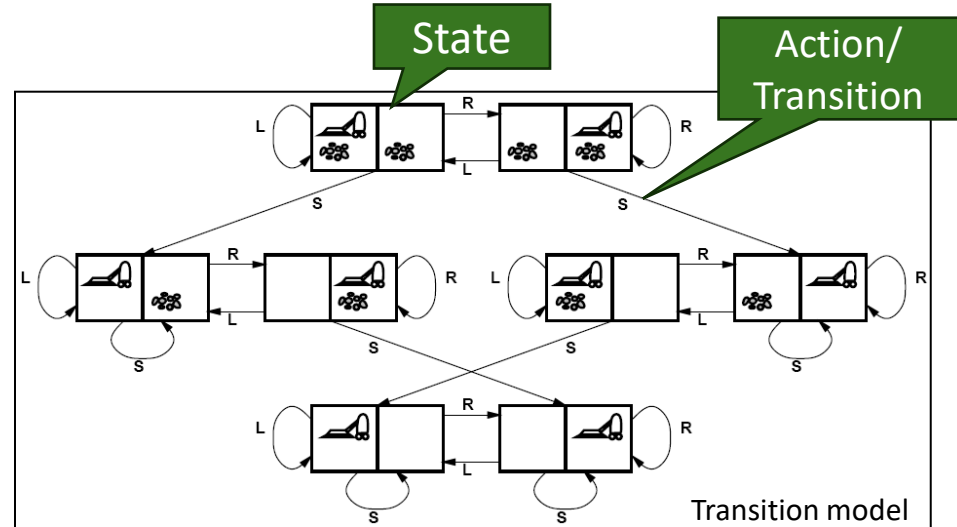
The set of all possible states is called the **state space  $S$** . This set is typically very large!

**Note:** We often construct atomic labels from factored state representations. E.g.: If the agent's state is the location  $x = 7$  and  $y = 3$ , then the atomic state label could be the string "(7, 3)". With the atomic representation, we can only compare if two labels are the same. With the factored state representation, we have more information. E.g., we can calculate the distance between the coordinates in two states.

# Transition Function

- How **the environment changes when actions are performed** is modeled as a discrete **dynamical system**.

- Example of a state diagram for the Vacuum cleaner world:



- States change because of
  - System dynamics of the environment (the environment evolves by itself).
  - The actions of the agent.
- Both types of changes are represented by the transition function written as

$$T: S \times A \rightarrow S$$

or

$$s' = T(s, a)$$

$S$  ... set of states

$A$  ... set of available actions

$a \in A$  ... an action

$s \in S$  ... current state

$s' \in S$  ... next state

# Old-school vs. Smart Thermostat



## Old-school thermostat

**Percepts**

**States**

**Transitions**

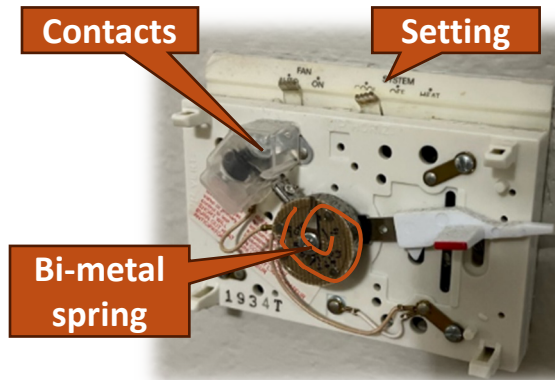
## Smart thermostat

**Percepts**

**States**

**Transitions**

# Old-school vs. Smart Thermostat: Solution



Set target temperature



Change temperature when you are too cold/warm.

Fluents model current situation

## Old-school thermostat

### Percepts

Setting: Cool, off, heat

Contact: Open, closed

### State

The agent uses no states (only reacts to the current percepts)

### Transitions

No transitions (has no states)

## Smart thermostat

### Percepts

#### Sensors

- Temp: deg. F
- Someone walking by
- Someone changes temp.

#### Internet

- Outside temp.
- Weather report
- Energy curtailment
- Day & time
- ...

### State

#### Factored description

- Estimated time to cool the house
- Someone home?
- How long till someone is coming home?
- Schedule
- ....

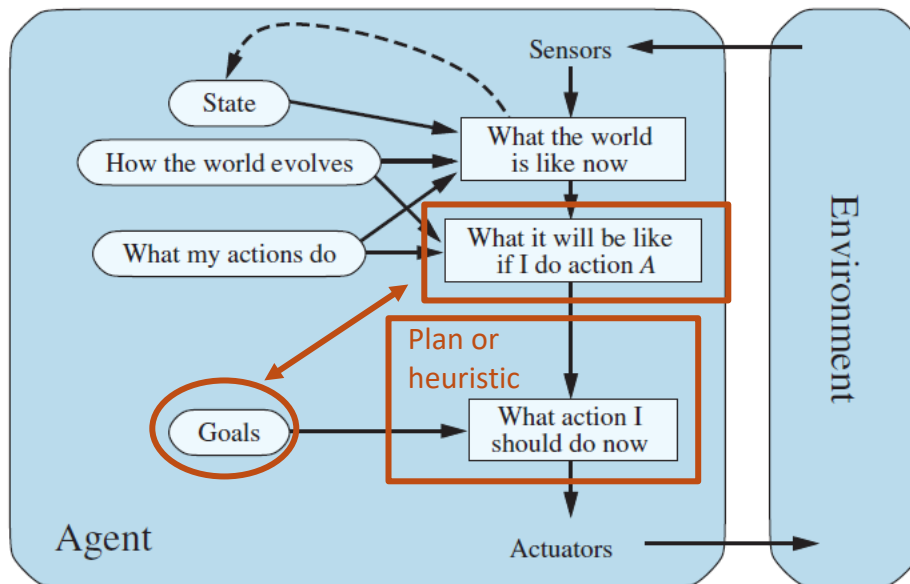
### Transitions

Many: E.g., Person walks by -> someone is home.  
Temperature changes -> estimated cool time changes

Actions or changes in the environment change the state

# Goal-based Agent

- The agent has the task of reaching a defined **goal state**, and then it is done.
- The agent needs to choose actions to move towards the goal. Subtypes:
  - **Greedy or heuristic goal-seeking agent**: Choose the next action to move towards the goal.
  - **Planning agent**: Use **search algorithms** to plan a sequence of actions that leads to the goal.
- Performance measure: the **cost to reach the goal**.



$$a = \operatorname{argmin}_{a_0 \in \mathcal{A}} \left[ \sum_{t=0}^T c_t \mid s_T \in S^{goal} \right]$$

Sum of the cost of a planned sequence of actions that leads to a goal state

The interaction is a sequence:  $p_0, s_0, a_0, p_1, s_1, a_1, p_2, s_2, a_2, \dots, s^{goal}$

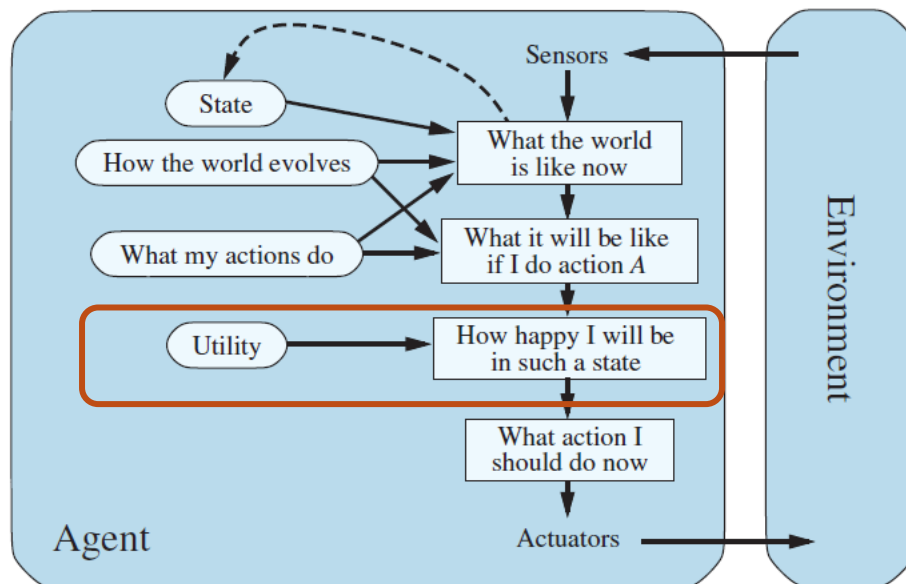
cost

**Example:** Solving a puzzle. What action gets me closer to the solution?



# Utility-based Agent

- The agent uses a utility function to evaluate the **desirability of each possible states**. This is typically expressed as the reward of being in a state  $R(s)$ .
- Choose actions to stay in desirable states.
- Performance measure: The discounted sum of **expected utility over time**.



$$a = \operatorname{argmax}_{a_0 \in A} \mathbb{E} \left[ \underbrace{\sum_{t=0}^{\infty} \gamma^t r_t}_{\text{Implements rational behavior: Utility is the expected future discounted reward}} \mid a_0 \right]$$

Implements rational behavior: Utility is the expected future discounted reward

**Techniques:** Markov decision processes, reinforcement learning

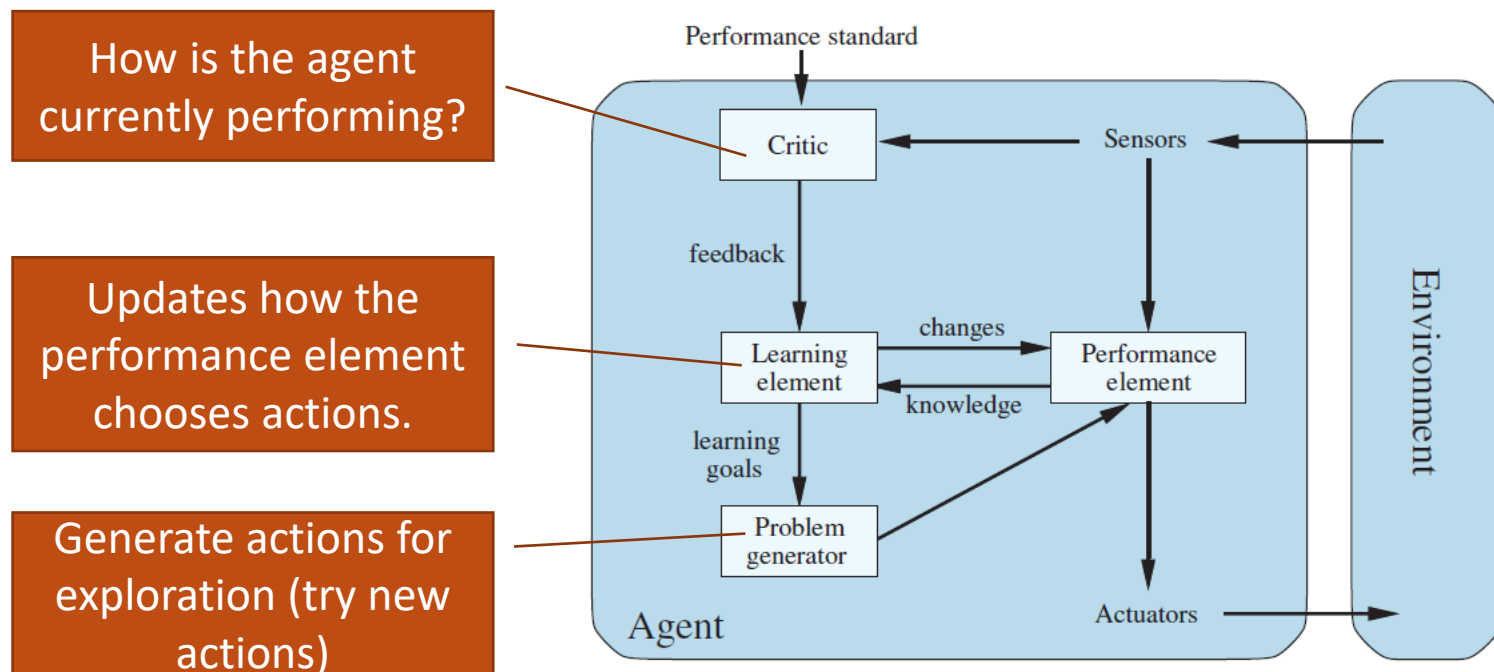
The interaction is a sequence:  $p_0, s_0, a_0, p_1, s_1, a_1, p_2, s_2, a_2, \dots$

$\underbrace{\hspace{10em}}_{\text{reward}}$

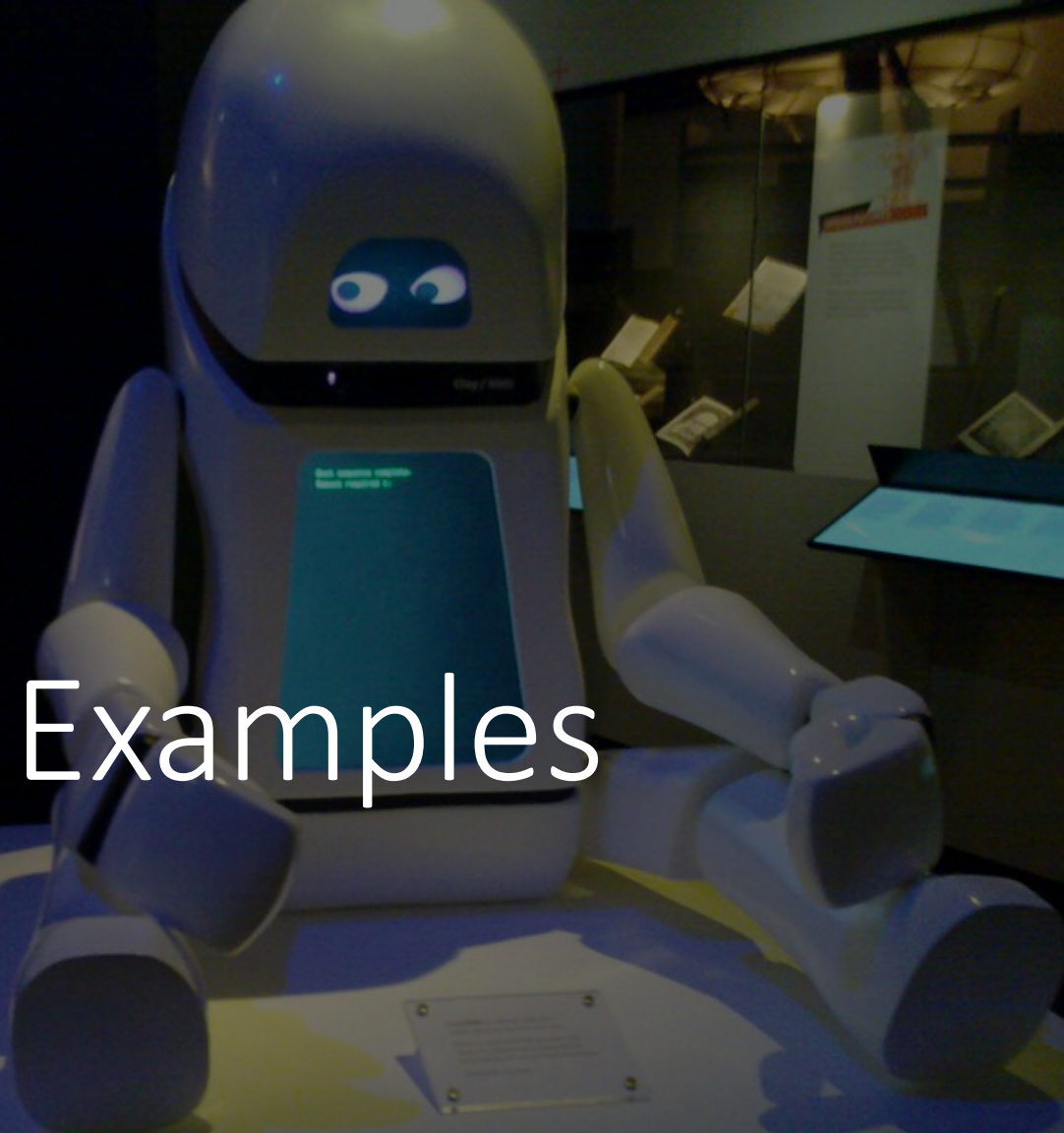
**Example:** An autonomous Mars rover prefers states where its battery is not critically low.

# Agents that Learn

The **learning element** modifies the agent program (reflex-based, goal-based, or utility-based) to improve its performance.



# Examples



# Smart Thermostat: What Type of Agent is it?



Change temperature when you are too cold/warm

Simple Reflex Agent?

Model-based Reflex Agent?

Goal-based?

Utility-based?

Learning?

## Smart thermostat

### Percepts

#### Sensors

- Temp: deg. F
- Someone walking by
- Someone changes temp.

#### Internet

- Outside temp.
- Weather report
- Energy curtailment
- Day & time
- ...

### States

#### Factored states

- Estimated time to cool the house
- Someone home?
- How long till someone is coming home?
- Schedule
- ....

# Example: Modern Vacuum Robot

Features are:

- Control via App
- Cleaning Modes
- Navigation
- Mapping
- Boundary blockers



iRobot's Roomba brand has become as synonymous with robot vacuum as Q-tips is with cotton swabs. The Wi-Fi-enabled Roomba 960 is ample evidence why. It turns a tiresome chore into something you can almost look forward to. With three cleaning modes and dirt-detecting sensors, it kept all the floor surfaces in our testing immaculate, and its camera-driven navigation and mapping were superb. Its easy-to-use app provides alerts and detailed cleaning reports. The ability to control it with Amazon Alexa and Google Home voice commands are just the cherry on top.

Source: <https://www.techhive.com/article/3269782/best-robot-vacuum-cleaners.html>

# PEAS Description of a Modern Robot Vacuum



**Performance  
measure**

**Environment**

**Actuators**

**Sensors**

# PEAS Description of a Modern Robot Vacuum: Solution



| Performance measure                                                                              | Environment                                                                                                                    | Actuators                                                                                                                                       | Sensors                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Time to clean 95%</li><li>• Does it get stuck?</li></ul> | <ul style="list-style-type: none"><li>• Rooms</li><li>• Obstacles</li><li>• Dirt</li><li>• People/pets</li><li>• ...</li></ul> | <ul style="list-style-type: none"><li>• Wheels</li><li>• Brushes</li><li>• Blower</li><li>• Sound</li><li>• Communicate to server/app</li></ul> | <ul style="list-style-type: none"><li>• Bumper</li><li>• Cameras/dirt sensor</li><li>• Laser</li><li>• Motor sensor (overheating)</li><li>• Cliff detection</li><li>• Home base locator</li></ul> |

**Note:** The PEAS description describes the task, but not how it is solved.



# What Type of Intelligent Agent is a Modern Robot Vacuum?



Is it learning?

Utility-based agents

Goal-based agents

Model-based reflex agents

Simple reflex agents

Does it collect utility over time? How would the utility for each state be defined?

Does it actively try to reach a goal state?

Does it store state information? How would states be defined (atomic/factored)?

Does it use simple rules based only on the current percepts?



Check what applies



# Example: Large Language Models



Default (GPT-3.5)



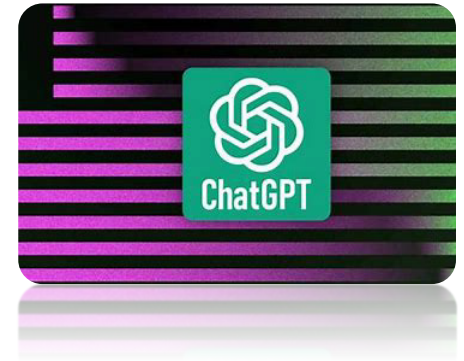
the sun is shining. It is



a beautiful day. The warm rays of sunlight illuminate the surroundings, creating a cheerful and inviting atmosphere. People are out and about, enjoying outdoor activities, and the clear blue sky adds to the overall sense of positivity. Whether you're taking a leisurely stroll, having a picnic, or simply basking in the sun's glow, it's a perfect time to appreciate the natural beauty and soak in the uplifting energy that the sunshine brings.

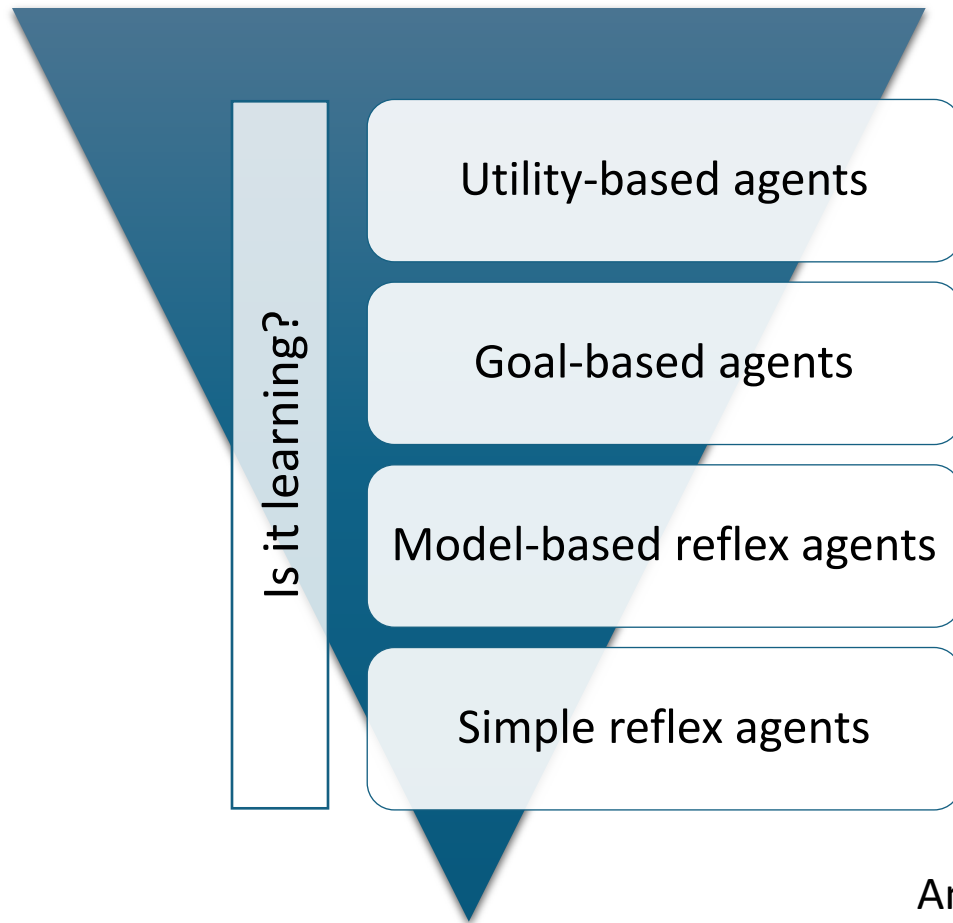


# PEAS Description of ChatGPT



| Performance measure | Environment | Actuators | Sensors |
|---------------------|-------------|-----------|---------|
|                     |             |           |         |

# What Type of Intelligent Agent is ChatGPT?



Does it collect utility over time? How would the utility for each state be defined?

Does it actively try to reach a goal state?

Does it store state information? How would the state be defined (atomic/factored)?

Does it use simple rules based on the current percepts?



Check what applies

Answer the following questions:

- Does ChatGPT pass the Touring test?
- Is ChatGPT a rational agent? Why?

We will talk about knowledge-based agents later.

# Intelligent Systems a Sets of Agents: Self-driving Car



It should learn!

Utility-based agents

Goal-based agents

Model-based reflex agents

Simple reflex agents

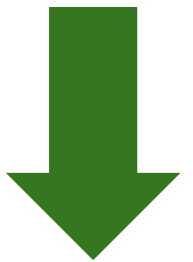
Make sure the passenger has a pleasant drive  
(not too much sudden breaking = utility)

Plan the route to the destination.

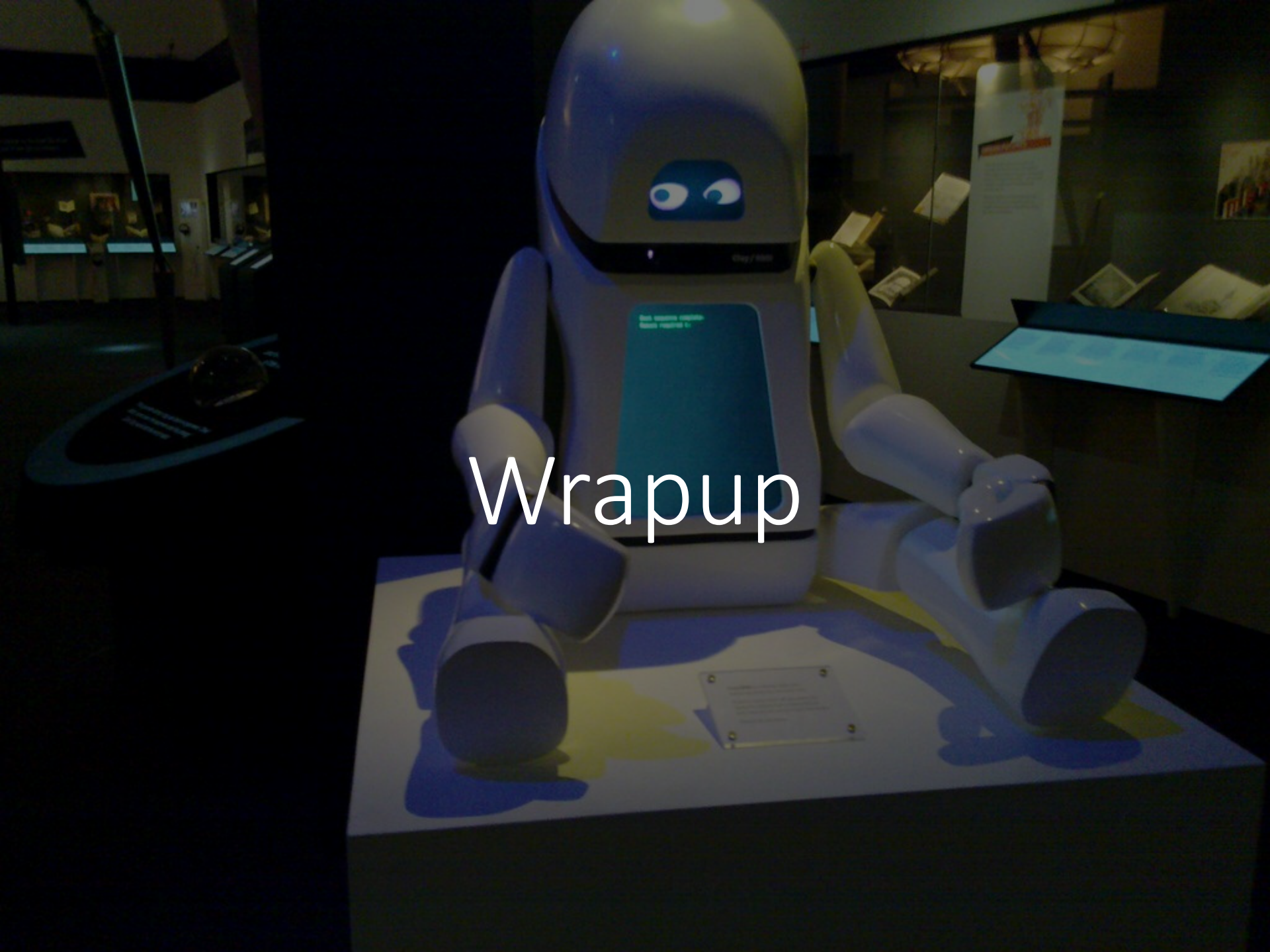
Remember where every other car is and  
calculate where they will be in the next few  
seconds.

React to unforeseen issues like a child  
running in front of the car quickly.

High-level  
planning



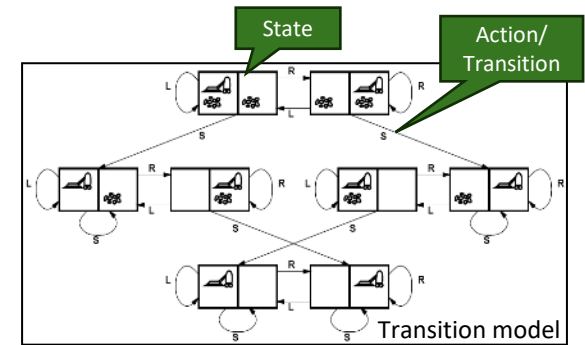
Low-level  
planning

A white humanoid robot with a friendly face, featuring large blue eyes and a small smile, is sitting on a white display table. The robot has a white body with blue accents on its arms and legs. It is positioned in a museum setting, with various exhibits and informational panels visible in the background. The word "Wrapup" is overlaid in white text on the robot's torso.

Wrapup



# Important Environment Types Revisited



State

**Fully observable:** The agent has access to the complete current **state** of the environment. It has deterministic percepts that are 100% reliable.

**vs.**

**Partially observable:** The agent's sensors provide incomplete or noisy information about the **state** of the environment. Noisy information means unreliable stochastic percepts (aka a stochastic sensor model)

Transition Function

**Deterministic:**  
Deterministic **transition function:** Changes in the environment are completely determined by the current state of the environment and the agent's action.

**vs.**

**Stochastic:**  
Stochastic **transition functions** lead to belief states, transition probabilities, and a Markov process.

**Known:** The agent knows the **transition function**.

**vs.**

**Unknown:** The needs to **learn the transition function** by trying actions.

# AI Areas

Intelligent agents inspire the research areas of modern AI

**Search for a goal**  
(e.g., navigation).

**Optimize functions**  
(e.g., utility).

**Stay within given constraints** (constraint satisfaction problem; e.g., reach the goal without running out of power)

**Deal with uncertainty**  
(e.g., current traffic on the road).

**Learn a good agent program from data and improve over time**  
(machine learning).

**Sensing**  
(e.g., natural language processing, vision)



# What You Should Know

---

- What an **agent function**  
 $action = f(percepts)$   
is and how it interacts with the environment.
- What are **states** and what is the **transition function**?
- How **environments** differ in terms of observability, uncertainty (stochastic behavior), and if the transition function is known.
- How to identify different **types of agents**.